

Cost Optimization of Adservice

Summary of a research spike to reduce
the cost of online boutique adservice
while maintaining current throughput

paulsbruce.github.io/app-optima
(github.com/paulsbruce/app-optima)



TL;DR

The good news: we found a way to reduce spend with Online Boutique

The bad news: reducing infra spend either means slowing down or speeding up

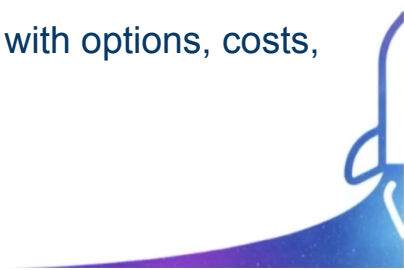
Homegrown

- Identifies marginal improvements to app/service performance
- Does not analyze holistic app architectural optima together (e.g. host/instance types)
- Does not tune autoscaling rules
- Savings would be anecdotal at best
- This effort alone cost ~\$2400
- Scaling out would cost Y1 ~\$40,000 and loss of a senior dev for a few months

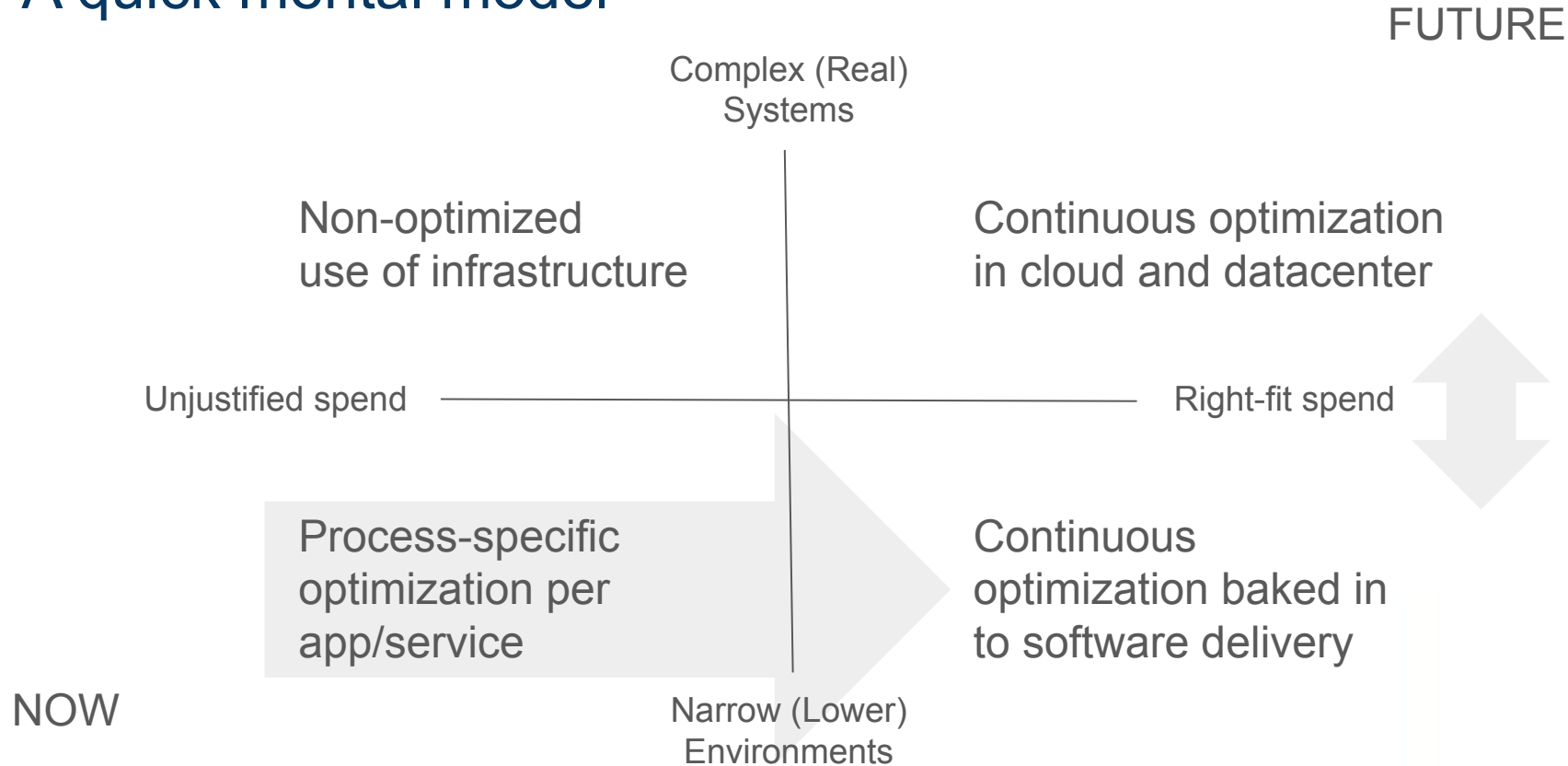
Recommendations

- Collect short-term evidence that cost savings is real, not just anecdotal
- Search for appropriate solutions (i.e. SaaS and vendor platforms); PoC some
- Build a likely rollout strategy for continuous optimization
- Come back (in a week) with options, costs, and a plan

We can't "slow down", so we have to "speed up".



A quick mental model



Context

- Business Imperative

- 2026 imperative for “no waste” → cost reduction → cloud optimization
- Find an optimization approach that can scale to the whole org

- Research Spike

- Boutique Store: thousands of shoppers, maintain throughput
- Adservice: custom text generation for ads, important for new products
- Lower environment for testing new configurations; 1/5th production sizing

- Expectations

- Create a solution to recommend configurations which:
 - Maintain current throughput (baseline)
 - *Lead to potential* lower costs in production
- Easy to implement in CI/CD pipelines; hands-off / no manual actions between steps



Methodology and Testing

- Most common workflow (New Visitor → Add Product(s) → Checkout)
- Ramp to 100 transactions per second over 5 minutes; prod (5x) is ~500TPS
- Experiment: Minimum 10 (acceptable) iterations...50 iterations over 7 hours
- (User) “Transaction” == Home, AddToCart, Checkout, etc.
- Multiple experiments at different times of day (b/c/ elastic infra resources)
- Host, services, and (test) client, and (Java) engine metrics collected
- “Acceptable” iteration
 - Less than 1% errors (industry standard for these kinds of load tests)
 - Throughput equal to (or greater than) baseline (first iteration on existing config)
- Scoring / Optima
 - Lower memory requests, then higher throughput, then lower response time

Assets

paulsbruce.github.io/app-optima
(github.com/paulsbruce/app-optima)

- Runs on any of our generic Gitlab build nodes easily (Python 3 dependency)
- Produces [HTML visual report](#) and programmatic JSON results

Best Recommendation

Metric	Baseline (Iteration 0)	Best (Iteration 49)
Memory Request	180.0	▲ 128.0
Memory Limit	300.0	▲ 256.0
Memory Use (Avg)	229.28	▲ 163.43
Heap	512	128
GC Type	UseSerialGC	UseG1GC
CPU Request	200m	▼ 250m
CPU Limit	300m	▼ 700m
CPU Use (Avg)	0.05	▲ 0.04
Throughput	95.56	100.12
Response Time (ms)	521.35	505.28
Error Rate (%)	0.0	0.0

Key Findings

- Minor improvements might help adservice cost (~10-15%)
 - Standardize garbage collector type to G1GC unless specific workloads require something else
 - Apply appropriate heap sizing that aligns with kubernetes memory constraints
 - Widen the CPU constraints so that momentary spikes don't cause resource pressure
- Higher demand in production likely to require more in-depth analysis
- Other components are more problematic than adservice (frontend, currency)
- (No) auto-scaling strategy in place; “constrained variable cost” vs. fixed
- To really optimize cost in production, consider infra pool config (mixed types)
- Cost of homegrown solution scale-out is ~\$40k (even with AI dev)
- Since we plan to roll this out across the board, we need “real” options

Cost of scaling-out the homegrown solution

What today's one-off "special instance" efforts took:

- 1 day research spike (environment and existing tooling familiarity)
- 1 day development (code, reporting, real-time vis via Grafana, small testing processes)
- 3 days validation (b/c long-running test processes)
- AI code assistance (~\$30 of tokens) to accelerate boilerplate work

What the scale-out would cost:

- Assumptions
 - Sr. dev salary ($\$100/\text{hr} \times 8\text{hrs}/\text{day}$) = $\$800/\text{day}$
- Per special instance: (2+1 days x salary/day) $\approx$ \$2500 per specialized instance
- Immediate needs:
 - Across 6 initial specialized instances (2 runtimes, 3 apps) = \$10,500
- 'Productization' costs (1 day/week @ $\$100/\text{hr} \times 8\text{hrs}/\text{day}$) + in-house support
 - $\$800/\text{week} \times 42$ days (first year needs) = \$33,600
- To in-house homegrown solution
 - First year (minimum estimate) = $\sim \$44,000$

Recommendations

- Progressively apply obvious minor improvements to prove them in production
 - Bring applicable tweaks over to other similar services as well; lessons learned...
 - Collect evidence that cost savings is real, not just an exercise
- Search for non homegrown solutions (i.e. SaaS and vendor platforms)
 - Immediate candidates:
 - Akamas
 - StormForge
 - Cast AI
 - Granulate
 - Develop a comparison matrix based on our organization's detailed requirements
- Build a reasonable rollout strategy for continuous optimization
- Come back (in a week) with options, real costs, and a better plan



Production Spend: Current vs. Improved (Projected)

- Lower (20%+overhead c8a.xlarge) vs. Prod (100%)
- On-demand 1-unit: c8a.large (2vCPU, 4GB RAM) = $\$0.10777 \times 24\text{hr} \times 365$; \$944/yr
- Production cost/yr: ~\$4720 currently; (5x units) current pool: vCPUs = 10, GB RAM = 20
- Given analysis and configuration tweaks, “Optimized”: vCPUs = 12, GB RAM = 16
- !!! Mix of smaller instances to make up pool; ~\$1457/yr
 - t3.small (2 vCPU, 2GB RAM @ \$0.0208/hr) x 4 = 8 vCPU, 8 GB RAM; 1-unit cost/yr = \$182/yr; 5x = \$728
 - t3.medium (2 vCPU, 4GB RAM @ \$0.0416/hr) x 2 = 4 vCPU, 8 GB RAM; 1-unit cost/yr = \$364/yr; 5x = \$728

	Lower (1-unit)	Prod (Current)	Optima Target	Prod (Optimized)	Savings	Effort-cost (custom dev)	Net Gain
c8a.xlarge	\$944/yr	\$4720/yr	+15%	\$4012	\$708/yr	\$2400	-\$984
Mixed t3	^^	N/A	?	\$1457/yr	\$3263	\$2400	\$863

Effort-cost often outweighs fine-tuning configuration + right-fitting infra savings!

Vendor Comparison Criteria and Initial Info

- Supports multiple runtime engines (Java, Node.js, Go, .NET)
- Supports Kubernetes and recommends auto-scaling configuration
- Supports both cloud and air gapped environments
- Supports using existing cloud monitoring solutions for production insights
- Provides recommendations for which cloud instance types to use

Automated some initial info collection...the most immediately actionable are:

- Akamas
- StormForge
- Turbonomic

...but FinOps visibility and performance optimization platform are different focuses. We should entertain options that don't try to do "all the things" poorly.



Next week

Work with Liana to:

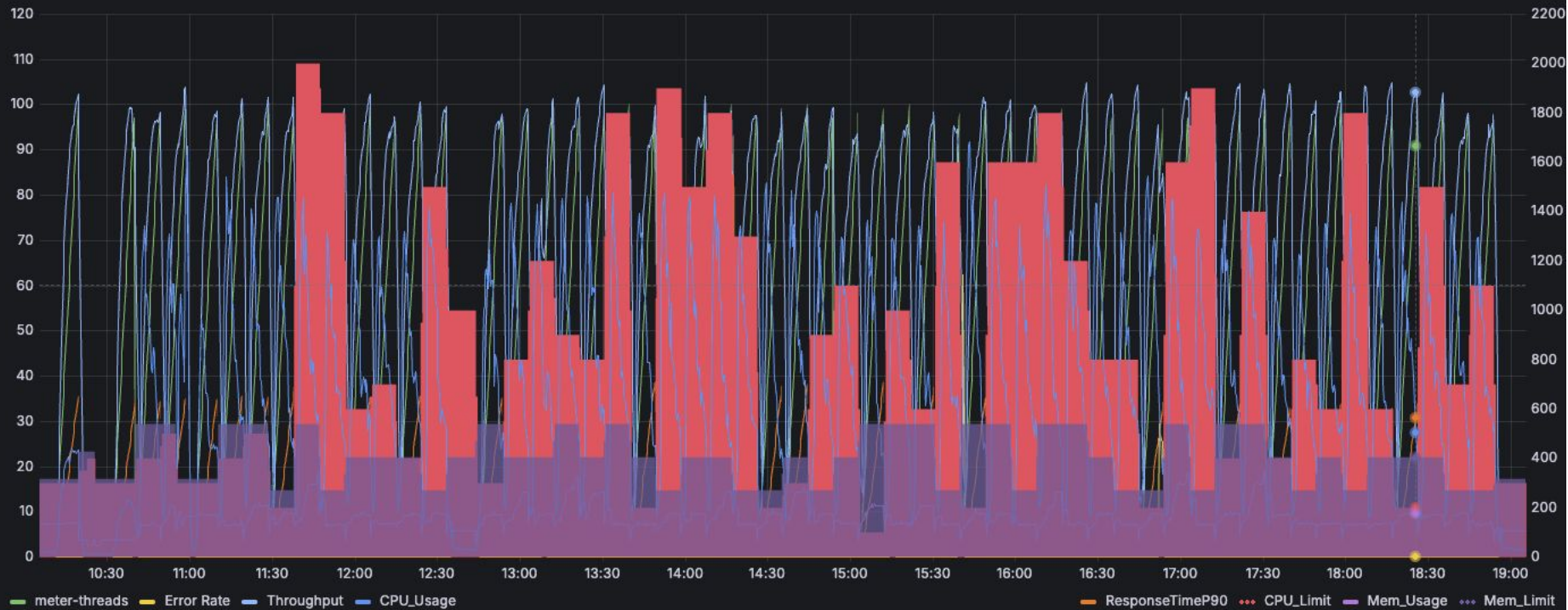
- Complete vendor analysis (verified capabilities matrix)
- Gather vendor costs and pricing models (non-negotiated)
- Develop a broader plan of action (incl. budget and rollout)

Also leverage James' relationships with partners and vendors.

This group to meet again for 30 minutes next week.



Testing Stats



Testing Stats

